

# ترکیب تحلیل ساختاری کد با استدلال مدل‌های زبانی بزرگ و معماری چندعامله برای کاهش هشدارهای کاذب در تحلیل آسیب‌پذیری کد

نگارنده: علی ناصری‌نیا

استاد راهنما: دکتر حمیدرضا شهریاری کاهکشی

# بیان مسئله و چالش‌ها



## راهکار خام و خطرات آن

استفاده مستقیم از LLM منجر به  
**توهم (Hallucination)** و تأیید بدون  
شواهد می‌شود.



## پیامد اصلی

بروز خستگی توسعه‌دهنده  
(Alert Fatigue) و کنار گذاشتن  
کامل ابزار امنیتی.



## (Static Analysis) تحلیل ایستا

ابزاری پرکاربرد اما مملو از هشدارهای  
نادرست (False Positives).

# راهکار پیشنهادی (سامانه codey-security)

ایده اصلی

تحلیل ایستا به عنوان «پیشنهاد دهنده»

+

هوش مصنوعی به عنوان «داور»

هدف

فیلتر کردن هشدارهای نادرست، بدون از دست دادن آسیب پذیری های واقعی.

نوآوری ها

✓ معماری چندعامله (جستجو گر و ارزیاب)

✓ استدلال منحصراً مبتنی بر شواهد ساخت یافته

# معماری کلی سامانه (معماری سه فاز)



# بسته شواهد و جلوگیری از نشت برچسب

## محتوای بسته شواهد

قطعه کد (شعاع ۸ خط)، فهرست فراخوانی‌ها (AST)، و مسیر جریان داده.

## (Label Leakage) چالش نشت برچسب

کدهای آزمایشی حاوی کلماتی مثل bad/good هستند که پاسخ را افشا می‌کنند.

## راه حل ما

ماژول پاک‌سازی کد: حذف حاشیه‌نویسی‌ها و بازنام‌گذاری متغیرهای افشاگر پیش از ارسال به مدل.



```
void process_data(char *bad_buffer var_1) {  
    char dest[50];  
    strcpy(dest, var_1); // Buffer Overflow  
}
```

# خروجی سه‌گزینه‌ای (نوآوری در تصمیم‌گیری)

پرهیز از تصمیمات سیاه و سفید (Zero-Sum).

نیاز به عامل انسانی



نامطمئن

(Uncertain)

شواهد برای قضاوت قطعی  
کافی نیست.



رد شده

(Rejected)

شواهد، بی‌خطر بودن کد را  
اثبات می‌کند.



تأیید شده

(Confirmed)

شواهد کافی برای وجود باگ  
در دسترس است.

# طراحی آزمایش‌ها و محیط ارزیابی

## مجموعه داده‌ها

### Juliet (C/C++)

زیرمجموعه ۶۰۰ فایل متعادل

### Python Benchmark

محک اختصاصی پایتون

### Design

دیتاست کدهای واقعی

## چهار پیکربندی آزمایش

A

فقط تحلیل ایستا (خط پایه)

C

ایستا + عامل زبانی

B

فقط عامل زبانی (LLM)

D

ایستا + عامل زبانی + شواهد  
ساختاری

# نتایج اصلی (کاهش چشمگیر هشدارهای نادرست)

۸۶%

کاهش هشدارها

نرخ هشدار کاذب از ۰/۰۵۰ به ۰/۰۰۷ رسید.

۳۶% ↑ ۶۰%

افزایش دقت (Precision)

ارتقای چشمگیر در شناسایی باگ‌های واقعی.



نرخ هشدار روی فایل سالم (FPR)



# تحلیل قیف تصمیم (مبادله دقت و یادآوری)

بهای کاهش هشدارها: افت یادآوری (Recall)

۰.۰۳۷ → ۰.۰۱۰

بسته شدن پرونده‌ها با وضعیت نامطمئن

۴۲%

دلیل افت Recall اشتباه مدل نیست، بلکه اعلام عدم قطعیت است.

نتیجه‌گیری: سیستم یک شکستِ ایمن (Safe-Fail) را به یک تأییدِ توهم‌گونه ترجیح می‌دهد.

# اثر شواهد ساختاری و بررسی‌های تکمیلی



## موتور قطعی پایتون

عملکرد درخشان در محک کنترل‌شده.

دقت **۹۶%**



## سرریز عدد صحیح

ضعف مطلق ابزارهای ایستا (یادآوری صفر).

توانایی بالای مدل زبانی در تشخیص (پیکربندی B).



## تأثیر شواهد ساختاری (AST)

کاهش عدم قطعیت.

۵ → ۱۱ تأیید صحیح

# نتیجه‌گیری و کارهای آینده

## نتیجه‌گیری نهایی

معماری codey-security نقطه کاری متعادلی برای سازمان‌های درگیر با هشدار کاذب است.

## گام‌های بعدی

- جایگزینی قطعه‌کد محدود با کل مسیر جریان داده (Dataflow Path).
- ساخت داشبورد انسانی برای بررسی سریع خروجی‌های «نامطمئن».
- استفاده از عامل‌های تعاملی با قابلیت پرس‌وجو (Query tool).

”

با تشکر از توجه شما

---

پرسش و پاسخ